

CRYPTOGRAPHY

Hash Functions: The Internet's Fingerprints

They verify your downloads, store your passwords, and hold blockchains together. A clear look at what cryptographic hash functions do and why they're irreversible.

Source: hash-functions-explained.md **Date:** 2026-06-03 **Author:** Ankit Gupta

Tags: hashing, passwords, security, blockchain

A cryptographic hash function is one of the most quietly important inventions in computing. It takes any input — a word, a file, an entire library — and produces a short, fixed-length fingerprint. Change the input by a single character and the fingerprint changes completely. This simple idea secures passwords, verifies downloads, and underpins every blockchain.

■ What a hash looks like

Feed text into a function like SHA-256 and you always get a 256-bit output, shown as 64 hex characters:

```
"hello" → 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824  
"hellp" → 7a3d3...completely different...e9c4
```

Notice: a one-letter change produces a totally different result. This is the **avalanche effect**, and it's essential.

■ The four properties that make it useful

1. **Deterministic** — the same input always gives the same hash.
2. **Fast** to compute in the forward direction.
3. **Irreversible** (preimage resistance) — given a hash, you can't feasibly work backward to the input.
4. **Collision-resistant** — it's practically impossible to find two different inputs with the same hash.

That third property is the magic: hashing is a one-way street. Multiplying is to factoring as hashing is to unhashing — easy one way, infeasible the other.

■ Where you rely on it every day

Password storage. Reputable services never store your actual password. They store its hash. When you log in, they hash what you typed and compare. If the database leaks, attackers get hashes, not passwords.

To make brute-forcing harder, systems add a random **salt** to each password before hashing (so identical passwords don't share a hash) and use deliberately *slow* hashes like bcrypt or Argon2.

File integrity. When you download software, the site often lists a hash. Hash the file you received; if it matches, the file wasn't corrupted or tampered with in transit.

Digital signatures. Rather than sign a huge document, you sign its hash — far faster, and just as secure thanks to collision resistance. (See [how RSA works](#).)

Blockchains. Each block contains the hash of the previous block. Alter any past transaction and its hash changes, which breaks every block after it — making tampering instantly detectable. That chained-hash structure is literally what “blockchain” means.

■ When hashes break

Security depends on collision resistance, and older functions have fallen. **MD5** and **SHA-1** are now broken — researchers can manufacture collisions — so they must never be used for security. The modern standards are **SHA-256** and **SHA-3**. The lesson: cryptographic strength has a shelf life, and migrating off weakened algorithms matters.

■ The takeaway

A hash function turns “is this exactly the thing I expect?” into a fast, reliable check — without ever revealing or storing the original. It's the humble workhorse behind passwords, downloads, signatures, and blockchains: a fingerprint for data that's easy to take and impossible to fake.

Curious how bcrypt's “work factor” keeps passwords safe as computers get faster? Ask below.